

可视计算与交互概论（VCL）

VCL

Author: Simson

Date: September 2026

目录

1 导论	1
2 VCL 基础知识和技能	1
2.1 颜色, 颜色感知和可视化	1
3 显示 (Display)	1
3.1 显示器的种类和发展历史	1
3.2 色域	2
3.3 像素标准	2
3.4 立体显示	2
4 图像 (Image Representation)	2
4.1 图像的表示	3
4.2 图形处理器 (GPU)	3
4.3 图像处理 (Image Processing)	4
4.3.1 Dithering	4
4.3.2 Filtering	5
4.3.3 Image inpainting	6
4.4 画图	7
4.4.1 直线的绘制	7
4.4.2 曲线的绘制	9
4.4.3 凸多边形的绘制	16
4.4.4 凹多边形与三角化	16
4.5 Warping	16
4.6 反走样 (Anti aliasing)	17
4.6.1 走样的发生原因	17
4.6.2 反走样处理·直线	18
4.6.3 反走样处理·图像	19
4.6.4 超采样技术	19
4.6.5 多重采样技术	20
5 几何 (Geometry)	20
5.1 多边形网格 Polygon mesh	21
5.1.1 Triangle Mesh 的存储	21
5.2 Subdivision Surface	23
5.2.1 Catmull Clark 细分算法	23
5.2.2 Loop subdivision	24
5.3 Mesh Parameterization	25

A 数学知识补充	26
A.1 Fourier 变换	26
A.1.1 二维 Fourier 变换的定义	27
A.1.2 二维 Fourier 变换的性质	27
A.1.3 Fourier 变换的运算	28
B Dithering 算法补充	28

1 导论

可视计算的定义：针对视觉内容生成与处理的所有计算机课学分制的总称（计算机图形学，计算机视觉，可视化，AR,VR）人类智能在感知，认知，决策，行动，感知的循环中逐渐积累。

重要课题：

- 物理世界仿真与模拟
- 计算机图形学
- 计算机视觉

2 VCL 基础知识和技能

2.1 颜色，颜色感知和可视化

人感知颜色靠的是对不同波长的光敏感的三种视锥细胞，这提供了颜色离散表达的基础。三种感光开展出 RGB 三色系统。颜色的匹配函数如下，是一个积分的办法。

$$\begin{aligned}R &= \int_0^{\infty} I(\lambda) \bar{r}(\lambda) d\lambda \\G &= \int_0^{\infty} I(\lambda) \bar{g}(\lambda) d\lambda \\B &= \int_0^{\infty} I(\lambda) \bar{b}(\lambda) d\lambda\end{aligned}$$

三种通道和原色的光由离散合成有两种机制：基于发光叠加相加混光（显示器发光原理）和基于白光吸收的相减混光（涂料和打印原理）。

计算机视觉基于人对光的感知。在人的感知之下，红色有前进的效应，蓝色会有看起来后退的效应。此即所谓色彩立体视觉。同时在人的认知下颜色也有冷色和暖色之分（这真的是课堂 PPT 里面的）。颜色信息在人眼中具有恒常性，不论在何种光照之下人对于颜色的认知不会发生大的改变。

当然这种逻辑感知也会带来相应的视觉错觉，譬如色诱导，使用补色可以让人自动脑补对应的颜色；再比如说边缘效应和移动错觉（动态立体视觉）。

人眼对于不同的光敏感性不同，对不同波段的光，施加灰度和模糊处理可以得到不同的认知效果（灰度指的是介于黑白之间的明暗程度）。

3 显示 (Display)

3.1 显示器的种类和发展历史

1. 矢量显示（阴极射线管）
2. 二维显示（电子束）
 - 像素：每个可以点亮的屏幕点

- 分辨率：品目像素格式
 - 纵横比：像素列数除以行数
 - 帧：显示的一个画面
 - 隔行扫描：保证亮度均衡
3. 彩色二维显示（彩色荧光幕，不同荧光剂分区涂布）
 4. 二维显示（LCD）：统一发光，用液晶部分控制光的偏振，调节亮度
 5. 二维显示（OLED）：有机发光半导体显示器。独立的 LED 单元。

3.2 色域

美国国家电视标准委员会的色域标准称为 NTSC。

- sRGB：微软，HP（72）
- ARGB: Adobe 颜色（设计软件）
- Rec.2020:4K(电视的标准)
- DCI-P3 : Apple 标准

3.3 像素标准

显卡的存储和刷新越厉害，达到 4K8K 就越容易。

3.4 立体显示

立体显示依据的原理是双目视觉。所有的立体显示技术基于人类的生理特征。最开始的显示技术是双目偏振片的 3D 显示。沉浸式的 3D 显示技术（AR,VR,XR）跟踪头显，裸眼 3D+ 眼睛跟踪。现有的技术向 MR-mix 方向进行发展。

裸眼 3D 技术：LCD 屏幕的方法：微柱透镜显示技术。利用视差屏障，令 OLED 屏幕上不同的像素点用不同角度进入人眼。

全息显示技术（Holographic display）：用干涉记录波前的信息（数字全息摄影），用衍射记录物光波场（受限于较小的范围）。

多模态显示技术：触觉显示（触觉输出），嗅觉显示（控制挥发的成分。。。?）

4 图像（Image Representation）

在二维空间中，每个点可以由四个参数表示。 $E(x, y, \lambda, t)$ 其中 λ 字母的含义为光的波长。从数学上来说图像本身是一个连续的代表（真实世界），在屏幕上离散化显示。VCL 的任务在于用离散方式进行还原。

4.1 图像的表达

图像显示的两个方法是矢量表示法 (Vector Representation, 对应 SVG 格式) 和光栅化表示法 (Raster Representation, 像素级存储)。

矢量表示法的精确度高, 用较小的空间, 不会受到分辨率影响 (可以用新的矢量计算适应不同屏幕不同大小) 多用于图标等规则图形。Raster 存储空间较大, 受限于分辨率, 但是可以表达更加不规则的图形。

4.2 图形处理器 (GPU)

GPU 有成百上千个简单的计算单元, 可以在同一时间并行运行大量程序, 比如计算每个像素的颜色, 而 CPU 一般只有个位数的可并行的计算单元。故而 GPU 显卡适合处理进行大量计算的图形编程任务。

Pixel 存储中有重要概念帧缓冲区 (Frambuffers for image processing and display), 指的是储存二维像素矩阵的内存大小。硬件以 60Hz 速度扫描帧缓冲区中的内容, 这也决定了显示器的种类。

同样存储彩色图片的时候, Fullcolor (RGB), 决定了显示器的颜色存储位数。一般来说真彩色图像的显示屏为 24-bit 位存储。如果颜色粗出空间较小, 色彩的丰富度就会下降。

gif 颜色存储的性质: Colormap 映射。对于每一个像素点, 存储一个 8-bit 常数, 映射到 RGB 颜色表格 Colormap, 根据映射进行显示。在预先选好的表格中进行查找和压缩。

几种图片的文件格式

常见图像文件格式的全称:

- **BMP**: Full color image (全彩色图像)
- **JPEG**: Joint Photographic Experts Group Format
- **TIFF**: Tagged-Image File Format
- **GIF**: CompuServe Graphics Interchange Format
- **PPM**: Portable PixMap Format (ASCII 或二进制)
- **EPS**: Encapsulated PostScript Format (ASCII)

	BITS PER PIXEL	FILE SIZE	COMMENTS
JPEG	24	small	lossy compression
TIFF	8, 24	medium	good general purpose
GIF	1, 4, 8	medium	popular, but 8-bit
PPM	24	big	easy to read/write
EPS	1, 2, 4, 8, 24	huge	good for printing

同时, 相较于 24-bit 的图像, 有的高级图像会膨胀到每个像素 96-bit, 有一些其他的通道来表示图像。

1. Alpha 通道 (8-bit): 透明度 (Transparency), 存储透明度常数, 存在前中后背景。

根据参数 α 计算像素点的表示: $b = (1 - \alpha)a_1 + \alpha a_2$

2. Z-buffer (16-bit): 保存像素点所对应的 3D 空间点的深度。判断在叠加一层的时候应该占据多少通道和 Alpha 信息。
3. 双通道表示: 既要保存图像的现有数据, 又要保存图像的旧信息, 方便修改。

4.3 图像处理 (Image Processing)

图像处理按操作的作用范围可以分为两类基本操作。**点处理 (Point Processing)** 逐像素地修改像素值, 输出仅是该像素自身像素值的函数; **滤波 (Filtering)** 的输出则是该像素周围 (通常是局部邻域 (local neighborhood) 内像素的函数, 因而需要用到邻域中其他像素的信息。我们也可以理解为一个逐点操作, 一个是区域化的操作 (图像抖动, 图像过滤, 区域补全)。

除这两类基本操作之外, 图像处理还包括以下其他课题:

- 图像变换 (Image transformation): 缩放 (resize)、形变 (warp)
- 图像压缩 (Image compression): 无损 (lossless) 与有损 (lossy)
- 图像编解码 (Image encoding and decoding): 用于高效传输 (efficient transmission)
- 图像分割 (Image segmentation)
- 图像抠图 (Image matting)
- 图像理解 (Image understanding): 进一步走向计算机视觉 (computer vision)
- 视频处理 (Video processing): 推广到图像序列 (a sequence of images)
-

4.3.1 Dithering

灰度图像频率的定义 灰度图像频率可以理解为灰度数值变化的剧烈程度, 数学表示通过对像素矩阵进行 DFT (傅里叶变换)。

设 $f(x, y)$ 为 $M \times N$ 的灰度图像, 其二维 DFT 及逆变换为:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$
$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

其中 $F(u, v)$ 为频域系数 (j 为虚数单位), (u, v) 为频率坐标: u, v 靠近原点对应低频, 即灰度缓变区域; 远离原点对应高频, 即边缘和细节。变换本身不损失信息, 逆变换可无损还原 $f(x, y)$ 。

Dithering 和噪声种类 抖动和量化研究的是有限空间下图片的表示, 比如要把灰度图像表示成二值的图像。此时如果直接使用 0.5 为分界划分每个 pixel。这时要给整体图片接一个整体的随机噪音 (random dithering)。原来是结构化的容易被人察觉的低频误差转换成频谱较为均匀的高频误差。(所谓频率指的是低频区间的扰动)

施加噪音的方法有两种: Random Noise 和 Blue Noise, Blue Noise 将误差的能量集中到高频的空间, 低频范围内噪声较小, 高频噪声能量较高, 能够得到更好的表示效果。

噪声的施加如果作为输入, 还会用于图像的模糊处理 (Gaussian blur), 均匀处理掉细节的误差之后 Blue Noise 会有更好的效果。

Dithering in Space 用空间换表示效果: 可以用多个像素点空间表示不同灰度, 这时可以将抖动的幅度变成 1-10 的连续序列。所有的输入像素对应一个输出的 pattern。这里 0-9 的 9 像素显示模式是经过设计的, 为了人尽量难以注意到细小的细节错误。这种方法叫作 Ordered Dithering。

数学上的方法就是使用存储抖动矩阵记录抖动的序列, 依次按照输入的归一化数值点亮像素点。

如果保持连续性不变化, 不改变显示分辨率, 进行有序抖动, 可以用一个 Dither Matrix 将图片扫描一遍, 像素点大于抖动阵的数值则保留, 小于则纯白, 可以实现同样分布的抖动。

Error Diffusion 每一次抖动都会制造误差, 可以通过这样的算法让每一次的误差逐步扩散到周边像素。Floyd 算法可以决定抖动误差如何进行分配, 让之前和之后的抖动误差有较好的平衡性和连续性。

如何在不同的建模和渲染场景中生成不同的合适的 Blue noise 是一个重要的话题, 在几乎所有的图像处理场景中 Dithering 都是必要的。

4.3.2 Filtering

Filtering 和图像的特征处理高度相关, 最广泛和最简单的应用是 Blurring 模糊。让高频信息逐渐减少, 剩下的集中在低频信息。基本的 Blurring Filter 使用模糊核将图像的局部进行适当的归一化。

最简单的模糊核是一个 3*3 矩阵, 无限维的卷积核相当于连续空间上的函数卷积

$$h(t) = f * g = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

相当于首先镜像然后向右平移。离散情况中卷积核在二维的像素图像上进行移动, 常用的卷积核 Laplace Kernel 在二维图像上卷积运算的数学表达如下, 其中 h 为卷积核, b 为输出, a 为输入。

$$b[x, y] = \sum_{u=-\infty}^{\infty} \sum_{v=-\infty}^{\infty} a[u, v]h[x - u, y - v]$$

Filtering 方法可以进行图像特征提取, 可以进行竖直方向和水平方向的有限差分, 从而提取不同的信息。边提取算子的数学表达如下:

$$\nabla a = \left(\frac{\partial a}{\partial x}, \frac{\partial a}{\partial y} \right), \quad |\nabla a| = \sqrt{\left(\frac{\partial a}{\partial x} \right)^2 + \left(\frac{\partial a}{\partial y} \right)^2}$$

将以上计算梯度的梯度信息取整数近似可以得到 Sobel edge Filter，是一个离散的非线性卷积核。

$$\frac{\partial}{\partial x} \Rightarrow \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

有限差分可以解决一阶微分，同时可以用 Laplace 算子求二阶微分提取出图像的边缘。离散化的 laplace 算子意思是显然的，就是对比某个像素点和周边的差异。laplace 算子的连续近似如下：

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$\frac{\partial^2 f}{\partial x^2} \approx f(x+1, y) - 2f(x, y) + f(x-1, y)$$

$$\frac{\partial^2 f}{\partial y^2} \approx f(x, y+1) - 2f(x, y) + f(x, y-1)$$

对应的离散卷积核为：

$$\mathbf{L} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

4.3.3 Image inpainting

图像补全的中心任务是填补 image 的某些缺失部分，让结果尽可能合理。这里的合理 (Plausible) 有以下含义：

- Spatial locality：和周围和谐
- Occam's Razor：不添加额外的非必要信息
- 梯度尽可能小

这种算法和计算过程常常称为 laplace 图像补全。这里优化函数的数学表示是

$$f = \arg \min_f \iint \|\nabla f\| d\Omega$$

这种技术还被应用于 Image cloning/insertion，图像克隆和拼接，算法可以完成前景和背景的融合。相比于之前的 Laplace 处理，这里要应用 Poisson 处理，目标是将另一张图片注入另一张图片，优化函数令当前的梯度和图片尽可能接近。前面的 Laplace 处理是 $\nabla g = 0$ 的特例 (背景是泛函分析中函数的变分法)

$$Purpose : \min_f \iint_{\Omega} \|\nabla f - \nabla g\|^2 dx dy$$

在计算机科学中本质是离散意义上 Poisson 和 Laplace 方程的求解。将梯度转换为差分，得到以下方程：

$$\frac{f_{i+1,j} + f_{i-1,j} - 2f_{i,j}}{h^2} + \frac{f_{i,j+1} + f_{i,j-1} - 2f_{i,j}}{h^2} = \rho^2$$

证明只需使用二元函数的 Taylor 展开，使用差分的方式进行计算，保留一次项。在实际应用中会注意到边界条件，如果边界条件是已知量（Dirichlet Boundary）则直接带入。离散化的 Laplace 算子形如如下表达式：

$$\nabla^2 f = \frac{f_{i+1,j} + f_{i-1,j} + f_{i,j+1} + f_{i,j-1} - 4f_{i,j}}{h^2}$$

对图片中所有的像素点使用 Laplace 算子，得到一个大型的线性方程组。使用 Jacobi 行列式进行迭代求解（Laplace Smoothing 算法）求解矩阵方程 $\mathbf{Ax} = \mathbf{b}$ ，有直接法（LU 分解），Jacobi 迭代法，Gauss-Seidel 迭代法。

Matting Matting 就是一般所指的抠图。抠图使用的方法使用掩码和前景。已知 Alpha 算子对图像进行混合，需要区分前景和背景。混合方程（compositing equation）为：

$$I_i = \alpha_i F_i + (1 - \alpha_i) B_i$$

对 RGB 三通道分别成立：

$$I_i^R = \alpha_i F_i^R + (1 - \alpha_i) B_i^R$$

$$I_i^G = \alpha_i F_i^G + (1 - \alpha_i) B_i^G$$

$$I_i^B = \alpha_i F_i^B + (1 - \alpha_i) B_i^B$$

其中 I_i 为观测像素值， F_i 为前景色， B_i 为背景色， $\alpha_i \in [0, 1]$ 为 Alpha 掩码。每个像素只有 3 个方程却有 F_i, B_i, α_i 共 7 个未知数。所以这个方程有无穷多个解。一种解决方法就是使用绿幕，令背景能够用已知的数值直接带入，减少未知数。这个方法的问题是，绿幕/蓝幕抠图由于漫反射可能会影响到前景。在当代也可以用 AI 方法辅助解决。

4.4 画图

基本任务：连续空间图像进行光栅化转换，用离散模式进行表示，研究这些表示方法的高效性和有效性。

4.4.1 直线的绘制

首先以简单的算法为例：假设要绘制直线 $y(x) = mx + b$ 。最直接的方法就是先建立坐标系，然后按照一定的步长遍历 x 轴，计算出 y 轴坐标之后进行取整。遍历 x 方向和 y 方向中函数增长较快的方向。因为 k 越小，浮点数的乘法越容易引发精度损失。

```
1 void line (int x0, int y0, int x1, int y1){
2     float y = y0;
3     float m = (y1 - y0) / (float) (x1 - x0);
4     int x;
5     for (x=x0; x<=x1; x++) {
```

```

6         float y= m*x + b;
7         draw_pixel(x, Round(y));
8     }
9 }

```

关注到上述 Naive 算法中出现了大量的浮点数运算，尤其是乘法，这是不经济的。利用上一步的数据，将乘积转换为加法迭代。关注到 $y(x+1) = y(x) + m$ ，将算法变为以下情况。

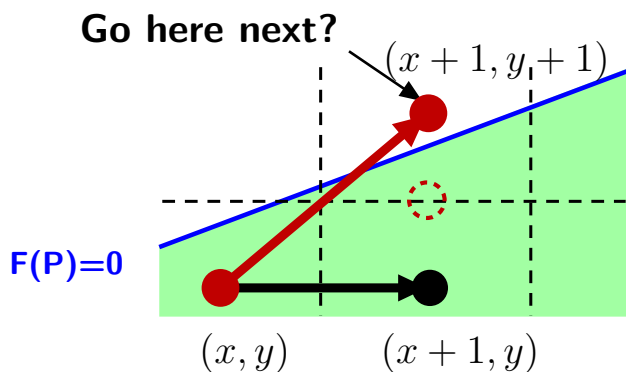
```

1 void line (int x0, int y0, int x1, int y1){
2     float y = y0;
3     float m = (y1 - y0) / (float) (x1 - x0);
4     int x;
5     for (x=x0; x<=x1; x++) {
6         draw_pixel(x, Round(y));
7         y += m;
8     }
9 }

```

把要求变得更加苛刻，可以直接避免浮点数运算，在累计过程中仅考虑整数，得到 Bresenham 绘制直线算法。这个算法的条件是，斜率范围处在 $(0, 1)$ 区间内。由于绘制的过程中，当前点如果是 (x, y) ，则下一个点要么横纵坐标都 +1，要么只有横坐标 +1。我们只需要判断直线的实际点离哪个更近，变成一个离散的二选一问题。

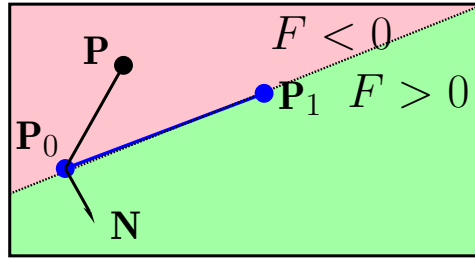
首先将直线的方程转换为隐函数 $F(x, y)$ ，能够根据函数值的符号判断点和直线的位置关系，判断两个点的方法就是判断中点的位置，即看 $F(x+1, y+\frac{1}{2})$ 的符号。



要想避免浮点数得到隐函数，可以借用直线上已知点 P ，关注直线法向量和 $\overrightarrow{PP_0}$ 的点乘结果，判断位置，从而构造出隐函数 $F(P)$

implicit function: $F(\mathbf{P}) = \mathbf{N} \cdot (\mathbf{P} - \mathbf{P}_0)$

$F = 0 \rightarrow \mathbf{P}$ is on L



由于最后只关心和 0 的关系，所以可以不用归一化直线斜率等信息，直接进行整数运算。由于代入点的时候道理是一样的，可以把斜率乘以 2，将向量的增量 $(1, \frac{1}{2})$ 也变成整数进行计算。

最终得到：

$$F(P) = 2N \cdot (P - P_0) \quad F(P + \Delta) = F(P) + 2N \cdot \Delta$$

是一个可迭代的整数运算。从而可以进行循环，得到该运算的代码，借用了点乘来构造方程：

```

1  int dx = 2*(x1-x0),
2  int dy = 2*(y1-y0);
3  int dydx = dy-dx;
4  F = 0 + dy - dx/2;
5  for (x=x0 ; x<=x1 ; x++) {
6      draw_pixel(x, y);
7      if (F<0) F += dy;
8      else {y++; F += dydx;}
9  }
```

4.4.2 曲线的绘制

上述算法只要知道了曲线方程的隐函数表示，可以用类似比较大小的手段画曲线，但是由于二次甚至三次方程的求解较为困难，所以很难优化到整数级别的运算难度。

对于圆这样的图形，每一次迭代可以同时画出对称的 8 个点，实现类似于并行计算的效果。

```

1  void draw_circle(int radius)
2  {
3      int x = 0, y = radius;
4      int d = 3-2*radius;
5      while (y>x) {
6          if (d<0) /* select East point next */
7              d += 4*x + 6;
8          else { /* select South-East point next */
9              d += 4*(x-y) + 10;
10             y--;
11             x++;
12             draw_8_pts(x,y); /* draws point in each octant */

```

```

13     }
14 }

```

曲线的表示方法 更加普遍的曲线绘制方法取决于曲线的表示方式，主要有两种：显式表示和隐式表示。

- Explicit 显式表达：函数表达式 $y = f(x)$ ，可以用带入的方法快速得到复杂曲线。
- Implicit 隐式表达：分形曲线，Signed Distance Fuction，解析几何方程

在曲线基础绘制中的一个主题是曲线的参数方程表示 (Parametric)。回顾圆的参数方程：

$$\begin{cases} x(t) = r \cos t, \\ y(t) = r \sin t \end{cases}$$

可以看到曲线方程具有较好的表示形式和稳定性，可以相应地降低计算量，表示的曲线范围也更广。并且数学上参数曲线的微分容易计算。

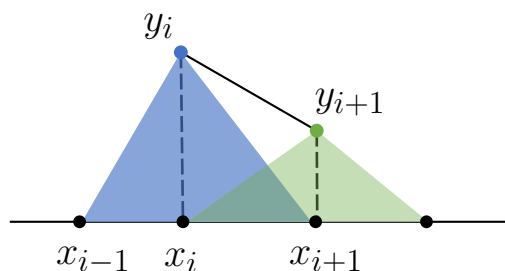
$$\mathbf{r}'(t) = \frac{d\mathbf{r}}{dt} = \begin{bmatrix} x'(t) \\ y'(t) \\ z'(t) \end{bmatrix}.$$

在明确了表示方法之后，我们可以发现曲线的绘制不能用直接的 lagarange 插值法得到高阶多项式，因为这样会使得计算难度过大。容易想到可以分段计算拼成光滑情况的多项式 (Piecewise Fuctions)，平衡连续性和可控性。由此发展出多种插值方法：

Linear Interpolation 线性插值在两个点之间直接绘制直线，一个线性插值函数记作 lerp，则：

$$\text{lerp}(t) = (1 - t)u_0 + tu_1$$

根据上述计算，构造插值的 Basis Fuction，将一个图像上所有点的插值规则集合在同样一个函数里面。已知插值方程为：



$$y = \frac{x_{i+1}-x}{x_{i+1}-x_i}y_i + \frac{x-x_i}{x_{i+1}-x_i}y_{i+1}$$

$$B_i(x) = \begin{cases} \frac{x-x_{i-1}}{x_i-x_{i-1}}, & x \in [x_{i-1}, x_i] \\ \frac{x_{i+1}-x}{x_{i+1}-x_i}, & x \in [x_i, x_{i+1}] \\ 0, & \text{else} \end{cases}$$

基函数是插值问题中更加广泛的概念，对于一个基函数 $B_i(x)$ ，满足以下要求（公理）：

$$B_i(x_j) = \delta_{ij} = \begin{cases} 1, i = j, \\ 0, i \neq j \end{cases}$$

Bezier Curve Interpolation 1962 年，法国雷诺汽车公司（Renault）的工程师 Pierre Bezier 提出了一种新的曲线表示方式：给定起点 u_0 与终点 u_1 ，再补充若干个额外的控制点，就能让一条光滑曲线被这些控制点“牵引”着从 u_0 走到 u_1 。

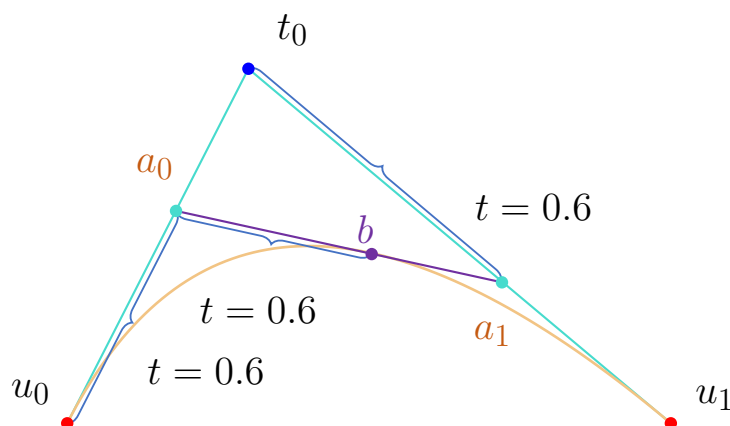
生成与动机 先考察最简单的配置：除两个端点之外只多给一个控制点 t_0 。构造完全建立在上一节的 lerp 之上，先在两条边上各取一个按同一参数 t 插值出来的中间点：

$$a_0 = \text{lerp}(u_0, t_0, t) = (1-t)u_0 + tt_0, \quad a_1 = \text{lerp}(t_0, u_1, t) = (1-t)t_0 + tu_1$$

再把 a_0, a_1 当作一对新的端点，做一次同样的插值：

$$b = \text{lerp}(a_0, a_1, t) = (1-t)a_0 + ta_1$$

固定 t 时 b 是曲线上的一个点；让 t 从 0 连续变到 1， b 扫过的轨迹就是所要求的那条曲线。下图取 $t = 0.6$ ：三条构造边上的花括号标注的其实是同一件事——每一轮插值得到的点都把所在的那条边按 $t : (1-t)$ 分成两段，所以三个花括号都写 $t = 0.6$ 。可以看到 b 恰好落在曲线上。



阶数提高时做法没有任何变化，只是“更多的线、更多次 lerp”：控制点增加到两个就是二次 (square) Bezier 曲线，增加到三个就是三次 (cubic) Bezier 曲线。一条三次 Bezier 有两个控制点，需要 $3 + 2 + 1$ 次 lerp——每做完一轮插值，点的个数就减少一个，所以这个过程天然递推的。

De Casteljau 迭代算法 上面这套做法称为 De Casteljau 算法（德卡斯特里奥）。给定参数 t ，它只有两步：

1. 对每一对相邻的控制点，按参数 t 计算它们的线性插值；
2. 用插值得到的结果作为新的控制点集合，重复上一步。

当控制点只剩下一个时递归终止，剩下的这个点就是参数 t 处的曲线上的点。写成递归函数即：

```
def bezier_recursive(points, t):
    # only one point left: recursion ends, this is the point on the curve
    if len(points) == 1:
        return points[0]
```

```

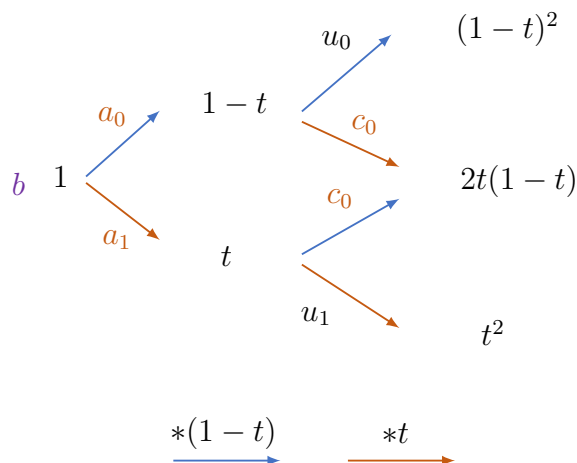
5
6 # otherwise: linear interpolation between every pair of neighbours
7 new_points = []
8 for i in range(len(points) - 1):
9     # linear interpolation: P = (1 - t) * P1 + t * P2
10    new_points.append((1 - t) * points[i] + t * points[i + 1])
11
12 # recurse on the reduced set of control points
13 return bezier_recursive(new_points, t)

```

把 lerp 的系数逐层乘开，就得到同一个结果的解析形式。以二次曲线为例（控制点在这里记作 c_0 ，也就是上文的 t_0 ）：

$$b = (1 - t)a_0 + ta_1 = (1 - t)^2u_0 + 2t(1 - t)c_0 + t^2u_1$$

中间项 $2t(1 - t)$ 的系数 2 来自两条路径的合并： c_0 既可以在 a_0 一侧乘 $(1 - t)$ 、再在 a_1 一侧乘 t 得到，也可以反过来。把每一条路径上的因子都画出来，就得到下面这棵树——蓝色箭头表示乘 $(1 - t)$ ，橙色箭头表示乘 t ，从 b 出发走到哪个端点，乘出来的就是那个端点的权重：



值得强调的是，尽管 Bezier 曲线有解析表达式，实际绘制时仍然使用 De Casteljau 算法：与解析表达式相比，它更加高效、数值也更加稳定。

解析表示：Bernstein 多项式与杨辉三角 把上面的过程继续推到三次，四个权重分别是

$$(1 - t)^3, \quad 3t(1 - t)^2, \quad 3t^2(1 - t), \quad t^3$$

系数 1, 3, 3, 1 恰好是杨辉三角（Pascal's triangle）的一行：

$$\begin{array}{ccccccc}
& & & & 1 & & \\
& & & & 1 & 1 & \\
& & & 1 & 2 & 1 & \\
& & 1 & 3 & 3 & 1 & \\
& 1 & 4 & 6 & 4 & 1 & \\
1 & 5 & 10 & 10 & 5 & 1 & \\
1 & 6 & 15 & 20 & 15 & 6 & 1 \\
1 & 7 & 21 & 35 & 35 & 21 & 7 & 1
\end{array}$$

因此这些权重函数也叫 Bernstein 多项式，一般地有

$$B(t) = \sum_k \binom{n}{k} t^k (1-t)^{n-k}$$

把权重乘回控制点，就得到 n 次 Bezier 曲线的解析表达式：

$$b(t) = \sum_{k=0}^n \binom{n}{k} t^k (1-t)^{n-k} u_k, \quad k = 0, 1, \dots, n$$

三次的情形写开就是

$$b(t) = (1-t)^3 u_0 + 3t(1-t)^2 u_1 + 3t^2(1-t) u_2 + t^3 u_3$$

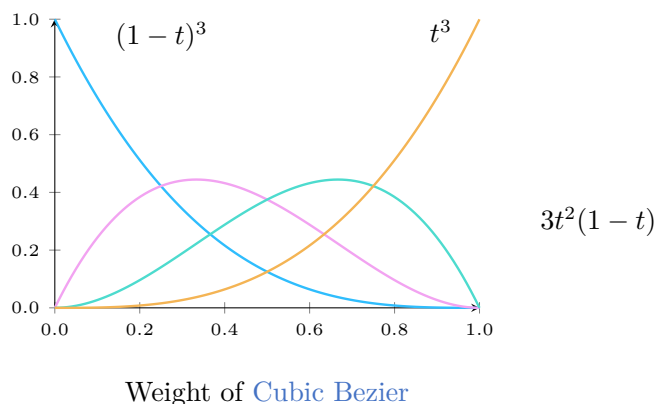
把它整理成矩阵乘积，可以更清楚地看到”权重”与”控制点”是分离的两部分：

$$b(t) = \boldsymbol{\tau}^T M_B \mathbf{u} = B(t)^T \mathbf{u}$$

其中

$$\boldsymbol{\tau} = \begin{bmatrix} 1 \\ t \\ t^2 \\ t^3 \end{bmatrix}, \quad M_B = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 \\ 3 & -6 & 3 & 0 \\ -1 & 3 & -3 & 1 \end{bmatrix}, \quad B(t) = \begin{bmatrix} (1-t)^3 \\ 3t(1-t)^2 \\ 3t^2(1-t) \\ t^3 \end{bmatrix}, \quad \mathbf{u} = \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ u_3 \end{bmatrix}$$

M_B 是只与次数有关的常数矩阵， $B(t)$ 则只与参数 t 有关，它被称为混合函数 (blending functions)。四条混合函数的图像如下。



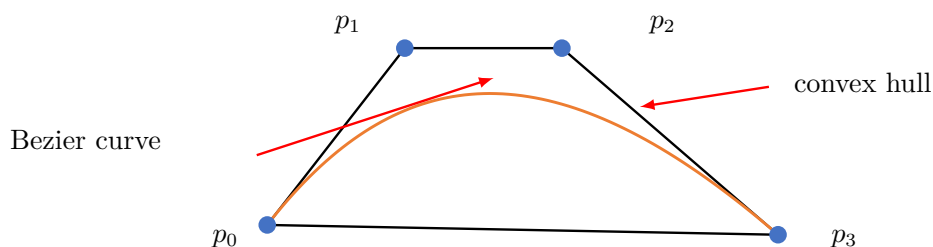
至此可以把 Bezier 曲线和上一节的 lerp 放在一起看：两者都是控制点的加权和，区别只在于权重函数。lerp 的权重是 $(1-t)$ 与 t ——也就是本节开头画过的那两个三角形；三次 Bezier 的权重则是上面这四条曲线。

性质 混合函数在端点处的取值决定了曲线在两个端点处的行为。对 n 次 Bezier 曲线，

$$B'(0) = n(u_1 - u_0), \quad B'(1) = n(u_n - u_{n-1})$$

也就是说，曲线在 u_0 处的切向就是 $u_0 \rightarrow u_1$ 的方向（这里的 u_1 指的是紧邻 u_0 的那个控制点）。借助这条性质可以光滑地拼接两条 Bezier 曲线：只要让接点两侧的一对控制点关于接点中心对称即可。

另一条性质来自 Bernstein 多项式本身：它保证了 Bezier 曲线一定落在其控制点的凸包（convex hull）之内——虽然这并不是最紧的界。因此即使我们不存储曲线上的所有数据，也不会偏离控制点太远。



B-splines B-样条曲线的绘制，主要算法是求解基函数。与 Bezier 的基函数是伯恩斯坦多项式不同，B-splines 曲线不设定两个控制点，而是通过多次的插值和直线向量的迭代一次得到 1 阶到 n 阶基函数。

数学本质 B-spline 曲线把分段参数曲线写成控制点的加权和：

$$\mathbf{b}(t) = \sum_{i=0}^m N_{i,n}(t) \mathbf{P}_i, \quad \mathbf{P} = \{\mathbf{P}_0, \mathbf{P}_1, \dots, \mathbf{P}_m\}$$

与 Bezier 的差别在于基函数的支撑集。Bernstein 多项式在整个 $[0, 1]$ 上都非零，所以一个控制点影响整条曲线，而片与片之间又毫无关联；B-spline 的基函数 $N_{i,n}$ 只在 $[t_i, t_{i+p+1}]$ 上非零，控制点因此只有局部影响，移动一个点只改动相邻的少数几段。基函数还满足单位分解

$$\sum_i N_{i,p}(t) = 1, \quad t \in [0, 1]$$

所以曲线仍然落在控制点的凸包之内。所有基函数的定义域取 $[u_0, u_m]$ ，通常令 $u_0 = 0$, $u_m = 1$ ，即闭区间 $[0, 1]$ 。

算法：Cox-de Boor 递推 基函数并不直接写出解析式，而是从 0 阶开始逐阶递推得到，这就是 Cox-de Boor 递推：

$$N_{i,0}(t) = \begin{cases} 1, & t_i \leq t < t_{i+1}, \\ 0, & \text{otherwise,} \end{cases}$$

$$N_{i,p}(t) = \frac{t - t_i}{t_{i+p} - t_i} N_{i,p-1}(t) + \frac{t_{i+p+1} - t}{t_{i+p+1} - t_{i+1}} N_{i+1,p-1}(t)$$

0 阶基函数是分段常数（方波），1 阶是三角波，阶数继续升高曲线逐渐光滑： p 阶基函数是 C^{p-1} 的。曲线的形状完全由节点序列（knot sequence）决定，节点均匀时称为均匀 B-spline。求值时给定 t 只有 $p+1$ 个基函数非零，逐层递推即可，代价与总控制点数 m 无关，这也是它比 Bezier 更适合表示长曲线的原因。

Cubic B-spline 取 $p = 3$ 。每条基函数跨越 4 个节点区间，于是 $[p_{i-1}, p_i]$ 这一段只由 4 个控制点决定：

$$\mathbf{p} = [p_{i-2}, p_{i-1}, p_i, p_{i+1}]^T$$

令 $u = (t - t_i)/(t_{i+1} - t_i)$ ，这一段可以写成与三次 Bezier 完全平行的形式

$$p(u) = \boldsymbol{\tau}^T M_S \mathbf{p} = \mathbf{b}(u)^T \mathbf{p}$$

其中

$$\mathbf{b}(u) = \frac{1}{6} \begin{bmatrix} (1-u)^3 \\ 4-6u^2+3u^3 \\ 1+3u+3u^2-3u^3 \\ u^3 \end{bmatrix}, \quad M_S = \frac{1}{6} \begin{bmatrix} 1 & 4 & 1 & 0 \\ -3 & 0 & 3 & 0 \\ 3 & -6 & 3 & 0 \\ -1 & 3 & -3 & 1 \end{bmatrix}$$

M_S 的每一行都不止一个非零元，一条基函数要跨过 4 段，这正对应它的局部支撑。三次 B-spline 具有 $C^{n-1} = C^2$ 连续性：函数值、一阶导与二阶导在连接点都连续，而且不需要像自然三次样条那样解方程组。

绘制时使用滑动窗口（sliding window）：每加入一个新的控制点， $\mathbf{p}$ 与 M_S 就向前滑动一段。每一段还可以改写成一条三次 Bezier 曲线，由

$$p(u) = \boldsymbol{\tau}^T M_B P_B = \boldsymbol{\tau}^T M_S P_S$$

得到 $P_B = M_B^{-1} M_S P_S$ ，也就是把 4 个 B-spline 控制点换算成 4 个 Bezier 控制点，之后就能直接用 De Casteljau 算法绘制。

把参数推广到两个方向就得到 B-spline 曲面 (patches)：

$$p(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 b_i(u) b_j(v) p_{ij} = \boldsymbol{\tau}^T M_S P M_S^T \boldsymbol{\tau}$$

形式上与 Bezier 曲面一样是 16 个控制点的加权和，但每片 patch 只覆盖定义域的 1/9。

4.4.3 凸多边形的绘制

凸多边形的判断并不容易，所以主要关注绘制三角形的算法。算法的基本原理是用一条直线自上而下进行扫描，一条一条地扫描，左右两侧的点由前述算法逐步确定。

对于一般的多边形可以进行三角化分割，归结到前面的算法。

具体来说，这套扫描转换流程需要依次完成以下几件事：先找出多边形的最高顶点与最低顶点，由此确定扫描线的起止范围；再把多边形的边沿着左右两侧分别组织成边表（左侧链与右侧链），按扫描线自上而下逐条处理；由于凸多边形每条扫描线只与边界相交两次，每条扫描线上只有一个待填充的区间 (span)，求出区间左右端点的横坐标 x_L, x_R 之后（端点也可以借助 Bresenham 算法顺带求出），把两者之间的像素全部填上颜色即可。相邻两条扫描线的端点之间满足

$$x_L += \frac{dx_L}{dy_L}, \quad x_R += \frac{dx_R}{dy_R}$$

所以端点的推进同样是增量式的，与前面画直线的迭代完全一致。

上面这些环节中任何一处处理不当，都会在相邻多边形的公共边上留下痕迹：如果两侧链的取舍或者端点的取整规则不一致，公共边可能被填充两次，也可能一次都没有被填充，于是出现重叠 (overlap) 或者裂缝 (crack)。

4.4.4 凹多边形与三角化

凹多边形的麻烦在于一条扫描线与它的交点可能多于两个，“每条扫描线只有一个区间”的前提不再成立。此时的做法是：求出该扫描线与多边形的全部交点，把它们按横坐标从左到右排序，再依次两两配对填充，这就是奇偶规则 (parity rule) ——排序后第 1、2 个交点之间是内部，第 3、4 个交点之间也是内部，奇数号交点为入点，偶数号交点为出点。也就是说，从某点向左引出的射线与多边形边界的交点个数为奇数时该点在内部，为偶数时在外部。

另一条思路是三角化 (triangulation)：先把凹多边形分割成若干个三角形，再对每个三角形套用前面凸多边形的扫描转换流程。这样做的好处是内层循环永远是简单的单区间填充，不要求交排序与奇偶判断；而只要共享同一条边的两个三角形对这条边的取整规则保持一致，拼接处既不会出现裂缝也不会重叠。代价是需要额外做一步三角化（对简单多边形可以用耳切法 ear clipping，逐个切掉凸耳）。两种做法本质上是在“逐扫描线求交排序”与“事先分割、简化内层循环”之间做取舍。

4.5 Warping

（课堂上面只是做了科普，真正的内容远远少于下面的表述，表述是 AI 完成的）

在 Image Warping 中, 进行图像的变形操作, 比如对图片进行拉伸, 扭曲, 控制。具体而言就是整体或者局部的坐标系变换。

常见的方法是网格方法, 将空间的网格进行拖动, 挪移等操作。在 PS 软件中, 实际上操作光标的中心是一个 kernel, 以相同的幅度和倍率进行平移和非线性形变。

对于 RGB 图像, 一般会进行颜色插值, 四边形和三角形的不同网格表示会导向不同的插值方法, 非线性插值和线性插值都会出现在 Warping 中。

具体地, 可以把图像看成定义在参数域上的函数: 源图的一个正方形区域由四个角点参数化——左上 (u_0, v_0) 、右上 (u_1, v_0) 、左下 (u_0, v_1) 、右下 (u_1, v_1) , Warping 就是把这个参数正方形映射成目标图中的任意四边形, 正方形的四个角落到四边形的四个角上, 源图里的参数点 (u_i, v_j) 则落到四边形中对应的位置。图中的向量 $\mathbf{u}$ 表示水平方向的映射, 整张图可以理解为参数域到目标域的一次坐标变换, 四边形的情形常取双线性映射, 更一般 (带透视) 的情形则是单应 (homography) 变换。

实现上通常采用反向映射 (inverse mapping): 遍历目标图像的每一个像素, 用逆变换求出它在源图中对应的参数坐标 (u_i, v_j) , 再按这个坐标到源图里取样。之所以不反过来做正向映射 (把源图的每个像素搬到目标位置), 是因为正向映射下目标像素未必被覆盖, 结果会出现空洞 (holes), 而多个源像素又可能落到同一个目标像素上造成重叠。

Warping 的典型用途是几何矫正: 把倾斜拍摄的画面 (仰拍的埃菲尔铁塔, 或者从侧面拍下来的蒙娜丽莎) 重新映射回正视图, 也就是所谓的 keystone 矫正; 也可以在图像上覆盖一张控制网格, 通过拖动网格的顶点做局部的非线性形变, 这与 PS 里的变形/液化工具是同一个思路。

颜色插值的问题随之而来。把由两个三角形拼成的正方形 (用红绿渐变示意) warp 成三角形时, 如果直接在屏幕空间对纹理坐标或者颜色做线性插值, 得到的是“非透视校正”的结果: 原本笔直的三角形公共边在结果图里会弯成一条曲线, 两块三角形的拼接处发生错位, 渐变也随之失真。原因是透视投影下屏幕上的等距并不对应纹理上的等距, 参数坐标要先除以深度 (齐次坐标里的 w) 才是线性的。正确的做法是透视校正插值 (perspective-correct interpolation): 对 u/w 、 v/w 、 $1/w$ 分别做线性插值, 最后相除还原出 u, v , 这样三角形的公共边才会保持笔直。

4.6 反走样 (Anti aliasing)

这一部分是图像处理中反走样的知识。图像发生走样的原因是进行采样的时候精度不够, 有一些信息丢失了, 因此导致图像模糊, 变形。

4.6.1 走样的发生原因

对于低频的信息, 间隔采样容易保留信息的大体走势。而对于高频的信息, 可能采样过程中精度不够, 则会发生混叠。划分低频和高频的标准是奈奎斯特准则, 即采样频率 f_s 应当大于图像频率 f 的两倍。当 $f_s > 2f$ 时候不发生走样, 当 $f_s < 2f$ 时会发生 aliasing

下面具体论证 Nyquist 条件 (Nyquist-Shannon Theorem):

以某一个频率进行采样, 等价于频域中脉冲谱和原频谱进行卷积, 产生 f_s 间隔的平移周期延拓。在应用中通常只绘一个基本周期内的频谱。如果把频谱在频率平面继续扩展, 会沿 f_x 和 f_y 方向以 f_s 为周期重复。如果采样频率过低, 则会使卷积之后的高频信号投影到低频的信号空间, 两个脉冲发生重叠, 在图片上会产生 aliasing

以图像为例子，一个高频信号会和低频信号产生相同的采样点，高频信号和低频叠加，就看不出信号的强度了。对于点阵等高频信息，在进行降采样的时候会出现 Moore 条纹。

4.6.2 反走样处理·直线

对于图片的反走样，要进行低通滤波，降低高频信号的比例，滤波结束之后可以办到防止走样。以直线的绘制为例，想要在绘制直线的时候防止图像走样，本质上是使用卷积核进行滤波，让显示直线的不同像素展现出不同的灰度。由于卷积是一个价格昂贵的计算，所以要尽可能降低 shading 的成本。

卷积核的设计由于直线是一维图像，可以直接把距离定义为信号强度。

面积采样：求交集面积就是一次卷积 理想直线是零面积的集合。只在像素中心判断“在不在线上” (0-1 采样)，等价于用冲激串对连续信号采样：频域中频谱以 f_s 为周期复制，而直线的硬边界含有任意高的频率，复制出来的高频落回低频区，就叠成锯齿。所以关键在于采样之前先做低通。面积采样把这一步直接算进了像素值里，不需要额外增加处理步骤。

给直线一个厚度 w ，得到一条带状区域，记它的指示函数为 $S(\xi, \eta)$ ，像素方框记作 Π ，则像素强度取两者的交集面积：

$$I[i, j] = \iint_{\text{pixel}_{ij}} S(\xi, \eta) d\xi d\eta = (S * \Pi)(x_i, y_j)$$

也就是说，求交集面积在数学上就是用 Π 这个盒核 (box filter) 对 S 做一次卷积，再在像素中心取值，滤波与采样合并成了一步。真正生效的核是 $\Pi_w * \Pi$ ，即两个方波的卷积，频域为 sinc^2 ，比单纯的 box (频域为 sinc ，旁瓣衰减只有 $1/|f|$) 要好，但仍不是陡峭的理想低通。

一维模型：覆盖率只依赖垂距 直线是一维图元，这一点让卷积可以大幅简化。一维情形下，宽 w 的带与单位区间的交叠长度只依赖带的中心到区间中心的偏移 d ：

$$T_w(d) = \max\left(0, \min\left(w, \frac{1+w}{2} - |d|\right)\right), \quad \int_{-\infty}^{\infty} T_w(d) dd = w$$

即一个梯形： $|d| \leq \frac{1+w}{2}$ 时饱和在 w ，到 $|d| = \frac{1+w}{2}$ 线性降到零 ($w = 1$ 时正好退化成 tent)。推广到二维，对直线 $y = mx + b$ ，像素中心到直线的垂距为

$$d = \frac{|mx_i - y_j + b|}{\sqrt{1 + m^2}}$$

对一条直带 (忽略端点与像素角点)，覆盖率仍然只由 d 决定。所以“把距离定义为信号强度”并非人为约定，而是因为覆盖率本来就只随垂距变化。注意 T_w 的积分等于 w 而不是 1，要当作合法的低通核使用，还需除以 w 归一化。

加权面积采样 把盒核换成可自由选择的权重函数 f ，就得到加权面积采样：

$$I[i, j] = \iint S(\xi, \eta) f(\xi - x_i, \eta - y_j) d\xi d\eta = (S * f)(x_i, y_j)$$

f 可以取任何形状，但要满足三条约束：

1. 支撑有限 (finite non-zero region): 每个像素只受邻近图元影响, 可以局部计算;
2. 随距离单调衰减: 越远的图元贡献越小, 避免相邻像素互相串色;
3. 全”面积”等于 1, 即 $\iint f = 1$: 归一化条件, 等价于频域的 $F(0,0) = 1$, 即直流增益为 1。只有这样, 一整片纯色区域采样后仍是原来的纯色, 不会被压暗或者提亮。

前两条只关乎”合理”, 第三条是硬性的。常见的四种核是 box、tent (Bartlett)、Gaussian 与 cone, 区别主要在频域的衰减速度:

$$\text{box} \rightarrow \text{sinc} (\sim |f|^{-1}), \quad \text{tent} \rightarrow \text{sinc}^2 (\sim |f|^{-2}), \quad \text{Gaussian} \rightarrow \text{Gaussian}$$

box 与 tent 的传递函数都带旁瓣, 截止频率以上的残留会以反相形式混回来, 表现为残余走样; Gaussian 没有旁瓣、衰减最快, 效果最好, 代价是支撑无限, 实现时必须截断, 而截断又会重新引入旁瓣, 工程上的取舍就在这里。这也说明加权为什么优于纯面积采样: 纯面积用的核是 $\Pi_w * \Pi$ (sinc^2 量级), 加权之后可以换成衰减更快的核。

幻灯片上写的是”blackness = area $\times$ $f(\text{distance})$ ”, 把精确交集面积与衰减权重分成两项相乘。严格说来加权面积采样是一次卷积 ($S * f$), 这个乘积形式是便于逐像素快速求值的近似, 方向一致但并不等价。

与超采样的关系 每像素做 $n \times n$ 超采样再取平均,

$$I[i, j] = \lim_{n \rightarrow \infty} \frac{1}{n^2} \sum_{k=1}^n \sum_{l=1}^n S(\xi_k, \eta_l)$$

正是上面那个积分的黎曼和, $n \rightarrow \infty$ 时收敛到精确值。两条路径的区别在于顺序: 面积采样解析地求出积分, 滤波在前、采样在后, 每像素 $O(1)$, 但只对能写出解析覆盖率的简单图元 (直线、多边形) 可行; 超采样数值地逼近积分, 采样在前、滤波在后, 通用于任何图元, 代价是 $O(n^2)$ 。这也回应了上面提到的成本问题: 直线之所以特殊, 正是因为它的覆盖率有闭式解, 所以能用面积采样把 shading 的成本压到最低。

4.6.3 反走样处理·图像

由于可能对于一张图片的处理需要多次进行低通滤波, 为了提高计算效率, 针对砖墙等有纹理的图像, 提出 MIP map 算法, 这个算法首先用一个 box 进行卷积平均, 做到 1 个像素为止, 然后把多个部分的压缩情况组合存储起来。经过预先计算形成金字塔之后, 根据所需的倍数和核的大小在层间进行线性插值 (光栅化)。

4.6.4 超采样技术

超采样的 kernel 有多种格式, 指的是在图像的变换中按照高于最终降伏网格的密度进行采样, 再把多个值合并成一个值得到灰度。

幻灯片中超级采样 (super-sampling) 的完整过程分三步: 先按屏幕分辨率采样, 每个像素只取一个点, 直线的取值非黑即白; 再提高分辨率采样, 采样点变密, 直线上出现灰度; 最后回到屏幕分辨率, 把每个像素内的多个采样值合并成一个值, 得到灰度。幻灯片左侧另外给出了源图与目标图的对应关系: 目标图中的一个像素 (红框) 在源图中对应一整块区域, 因此需要在该区域内取多个采样点。

幻灯片列出的 kernel 格式依次是：1 sample, 1×2 sample, 2×1 sample, Quincunx, 2×2 grid, 2×2 RGSS, 4×4 checker, 8 rooks, 4×4 grid, 8×8 checker, 8×8 grid。每一种格式都用图标画出了子采样点在像素内的位置，右侧并排给出该格式绘制同一条直线（一个填充三角形与一个线框三角形）的结果。

4.6.5 多重采样技术

多重采样是针对图形而言的，每个像素布置 4 个子采样点，像素中心不在三角形内部，但部分子采样点被三角形覆盖，该像素仍然会被标记为需要处理。相对超采样，MSAA 对该像素仅仅计算一次，不对每个采样点都进行单独着色，然后计算出结果颜色。

幻灯片对比的前提是普通单采样：单采样只看像素中心是否落在三角形内。

以每个像素布置 2×2 共 4 个子采样点为例，幻灯片把像素分成两类：

- 内部像素：4/4 被覆盖，着色 1 次，输出 $= C$ ；
- 边界像素：像素中心在三角形外，但有 3/4 的子采样点被覆盖，该像素仍会被标记为需要处理。若无反走样，该像素可能直接被丢弃。

边界像素的处理流程分三步：覆盖检测（统计 4 个子采样点中有 3 个被覆盖） $\rightarrow$ 只计算一次颜色 C （对该像素的三角形只着色一次） $\rightarrow$ 按覆盖率混合（根据覆盖率与背景进行颜色混合）。于是覆盖率 $= 3/4$ ，最终颜色 $= (3/4) \times C$ 。

直观理解：三角形内部的处理方式几乎与单采样相同；三角形边界则根据子采样覆盖比例平滑边缘。MSAA 在边界处效果接近 SSAA，但着色开销更低。

MSAA 与 SSAA 的对比：SSAA 每个子采样点都单独着色，质量高，开销大；MSAA 主要对可见性做多重采样，对颜色通常只着色一次，开销更低。

幻灯片最后问 MSAA 能否同时改善两者，结论是要分成两类走样来看：

- 几何反走样：圆形等物体的外轮廓属于几何 aliasing，光栅化时会出现边缘锯齿，MSAA 擅长处理覆盖率/边界锯齿；
- 着色反走样：镜中的天空、云朵、窗格等在一个三角形内部会有高频的颜色、纹理和光照变化，也会产生 aliasing，即使远离物体轮廓边缘；这类高频反射、纹理、光照变化需要着色采样/滤波。

5 几何 (Geometry)

这里计算机图形学的研究范围拓广到了三维，主要研究 3D 图像的建模问题。首先是介绍性的内容：如何在计算机上编码和表达一个三维物体？首先是显式表达方法：

- Explicit: 显式表达，计算效率更高。
 1. Point Cloud (点云)
 2. Polygon mesh (多边形网格，最经典和重要)
 3. Subdivision surface
- Implicit: 隐式表达，适合表达更需要细节的内容

以上所有方法都是建立三维到二维的映射并进行二维存储。

5.1 多边形网格 Polygon mesh

5.1.1 Triangle Mesh 的存储

在计算机图形学 (Computer Graphics) 里, .obj 通常指 Wavefront OBJ 文件, 是一种非常经典的三维模型文件格式。下文以 obj 格式为例。

多边形网格 (polygon mesh) 的通用表示是三张表: vertex table (顶点表)、edge table (边表) 和 polygon-surface table (面表)。顶点表逐个记录顶点坐标, $V_1 : x_1, y_1, z_1$ 到 $V_5 : x_5, y_5, z_5$; 边表记录每条边的两个端点, $E_1 : V_1, V_2$, $E_2 : V_2, V_3$, $E_3 : V_3, V_1$, $E_4 : V_3, V_4$, $E_5 : V_4, V_5$, $E_6 : V_5, V_1$; 面表记录每个面由哪些边围成, $S_1 : E_1, E_2, E_3$, $S_2 : E_3, E_4, E_5, E_6$ 。另外, 多面体上还有欧拉定理: $v - e + f = 2$ 。

最简单的存储三角形的方法是 Triangle Soup, 按照三角形的编号依次存储。三个点有序编号, 可以看出三角形的三维朝向。例如

$$\begin{aligned} \text{tris}[0] &= \{(x_0, y_0, z_0), (x_2, y_2, z_2), (x_1, y_1, z_1)\} \\ \text{tris}[1] &= \{(x_0, y_0, z_0), (x_3, y_3, z_3), (x_2, y_2, z_2)\} \end{aligned}$$

这个方法的问题是一个点可能有好几个编号, 非常混乱。

稍微高效一点的是 obj 的 list of vertices 和 list of indexed triangles。存储所有顶点的坐标, 以及用下标储存的面的列表: $\text{verts}[0], \text{verts}[1], \dots$ 依次是 (x_0, y_0, z_0) 、 (x_1, y_1, z_1) 、 (x_2, y_2, z_2) 、 (x_3, y_3, z_3) 等等, $\text{tInd}[0] = 0, 2, 1$, $\text{tInd}[1] = 0, 3, 2$ 。它比 Triangle Soup 复杂, 但共享顶点位置信息, 内存占用更小; 改动一个顶点在三维空间中的位置, 所有用到它的多边形都会跟着移动, 网格的完整性因此得到保证。

上述方法在读取和翻译时, 由于后面的边集合每个都用形如 (起点, 终点, 所在三角形) 的三元组表示, 依据三角形对三元组进行排序, 依次找出三角形对应的边和公共边集合。以找三角形的邻居为例, 先把顶点位置写成一个向量

$$\mathbf{x} = \begin{bmatrix} \mathbf{x}_0 \\ \vdots \\ \mathbf{x}_i \\ \vdots \\ \mathbf{x}_{10} \end{bmatrix} \in \mathbf{R}^{33}$$

(共 11 个顶点, 每个 3 个坐标)。三角形列表是一串下标三元组: $\{(0, 1, 2), (2, 1, 4), (2, 4, 3), \dots\}$ 。把每个三角形的三条边都写成 (起点, 终点, 所在三角形) 的三元组, 得到 triangle edge list:

$$\{\{0, 1, 0\}, \{1, 2, 0\}, \{0, 2, 0\}; \{1, 2, 1\}, \{1, 4, 1\}, \{2, 4, 1\}, \{1, 4, 2\}, \dots\}$$

(最后一位是三角形编号)。对它 sort and remove, 同一条边的记录就排到了一起, 得到按边组织、仍带三角形编号的 edge list:

$$\begin{aligned} &\{\{0, 1, 0\}, \{0, 2, 0\}, \{0, 2, 4\}, \{0, 9, 4\}, \{0, 9, 5\}, \{0, 10, 5\}, \dots \\ &\{1, 2, 0\}, \{1, 2, 1\}, \{1, 4, 1\}, \{2, 3, 2\}, \{2, 3, 3\}, \{2, 4, 1\}, \{2, 4, 2\}, \dots\} \end{aligned}$$

以及只留端点对的 edge list: $\{\{0, 1\}, \{0, 2\}, \{0, 9\}, \{0, 10\}, \{1, 2\}, \{1, 3\}, \{1, 4\}, \dots\}$, 由此得到相邻三角形列表 neighboring triangle list: $\{\{0, 1\}, \{1, 2\}\}$, 即边 $\{0, 1\}$ 由三角形 0 和 1 共用, 边 $\{1, 2\}$ 由三角形 1 和 2 共用。反过来, 如何找顶点的邻居?

针对以上方法的局限性，发展出了半边的数据结构。每一条边可对应双方向两个三角形。

其中的关键想法是：把每条边拆成两条方向相反的有向半边（half-edge），一条归它一侧的面，另一条归另一侧的面，两条半边互为 twin（孪生）。这样一来网格中就不再有孤立的元素：顶点、边、面各自指向自己的一条半边，互为 twin 的两条半边把相邻的两个面“粘”在一起，从任何一个元素出发都能顺着这些指针走遍整个网格。

单看一条边：它被拆成方向相反的两条半边，一条属于它左侧的面，一条属于它右侧的面，二者互为 twin，边本身只是装这两条半边的容器。同一条半边在它所属的面内还有下一条半边 next，沿着 next 就能绕着这个面走一圈。

四个结构体的定义如下：

```

1 struct Halfedge {
2     Halfedge *twin;    // 孪生半边
3     Halfedge *next;    // 同一个面内的下一条半边
4     Vertex   *vertex;  // 指向的顶点
5     Edge     *edge;    // 所属的边
6     Face     *face;    // 所属的面
7 };
8
9 struct Vertex {
10     Vec3      pos;      // 顶点位置
11     Halfedge *halfedge; // 该顶点的一条半边
12 };
13
14 struct Edge {
15     Halfedge *halfedge; // 该边的一条半边，另一半由 twin 找到
16 };
17
18 struct Face {
19     Halfedge *halfedge; // 该面的一条半边
20 };

```

有了这些指针，遍历的代码可以写得很短。第一个例子是遍历一个面的所有顶点：

```

1 Halfedge* h = f->halfedge;
2 do
3     do_work(h->vertex);
4     h = h->next;
5 while (h != f->halfedge);

```

从面 f 的某条半边出发，每次处理当前半边指向的顶点 h->vertex，再沿 next 走到同一个面内的下一条半边，绕一圈回到起点为止。整个过程只用到 next 一个指针。

第二个例子是遍历一个顶点周围的所有边：

```

1 Halfedge* h = v->halfedge;
2 do
3     do_work(h->edge);
4     h = h->twin->next;

```

```
5 while (h != v->halfedge);
```

从顶点 v 的某条半边出发, 先处理它所在的边 $h \rightarrow \text{edge}$, 再走 twin 跨到相邻的那个三角形、接 next 转回同一个顶点, 如此绕着顶点走一圈。halfedge、twin、next 三个箭头依次把顶点周围的边串了起来。

参考: <https://jerryyin.info/geometry-processing-algorithms/half-edge/>

需要补充的是相比三角形网格, 四边形网格容易变得平滑, 且容易进行存储。很显然你 QuadMesh 的存储方法非常简单, 但是想要得到平滑的表面需要多得多的空间。

5.2 Subdivision Surface

中文翻译细分曲面, 指的是通过不断增加控制点, 不断细分插值, 由从一个比较粗糙、低多边形的控制网格出发, 通过不断“切小 + 调整顶点位置”, 逐渐逼近一个光滑曲面。主要介绍两种算法

5.2.1 Catmull Clark 细分算法

这个算法中的很多权重来源于工程经验, 没有什么特别的依据和数学定理。这个算法主要是用来根据新加入的控制点更新顶点的。

一次细分做的事情是: 往原来的每个面里放一个新的点 (face point), 往每条边上放一个新的点 (edge point), 再把原来的每个顶点挪到一个新的位置 (vertex point), 最后按规则把这些点连起来。于是原来的一个四边形面被切成四个, 网格变密了一层, 重复做下去就越来越光滑。

先约定记号。设某个四边形面为 f , 它环绕一周的四个顶点为 v_1, v_2, v_3, v_4 ; 某条边的两个端点为 v_1, v_2 , 它两侧的两个面为 f_1, f_2 ; 某个顶点记作 p , 它周围有 n 个面和 n 条边, 这 n 个面记作 $f_1, \dots, f_n$, 和它相连的 n 条边的中点记作 $m_1, \dots, m_n$ 。注意 m 只是边的中点 (midpoint), 和要新增的 edge point 不是一回事。

Face point: 取该面所有顶点的平均, 也就是这个面的重心

$$f = \frac{v_1 + v_2 + v_3 + v_4}{4}.$$

Edge point: 不只是取边的两个端点, 还要把它两侧的面点也算进来

$$e = \frac{v_1 + v_2 + f_1 + f_2}{4}.$$

Vertex point: 原来的顶点 p 被更新为

$$v = \frac{1}{n} \left[\frac{f_1 + f_2 + f_3 + f_4}{4} + 2 \frac{m_1 + m_2 + m_3 + m_4}{4} + (n-3)p \right],$$

这是图中画的 $n=4$ 的情形, 写成一般形式即

$$v = \frac{1}{n} \left[\frac{1}{n} \sum_{i=1}^n f_i + \frac{2}{n} \sum_{i=1}^n m_i + (n-3)p \right].$$

括号里三项分别是周围 face point 的平均 (权重 1)、周围边中点的平均 (权重 2) 和顶点自己的原位置 (权重 $n-3$), 最后整体除以 n 。三项权重之和为 $1+2+(n-3)=n$, 再除 n 恰好归一, 所以新的顶点位置是这几项的加权平均, 权重本身则来自工程经验。

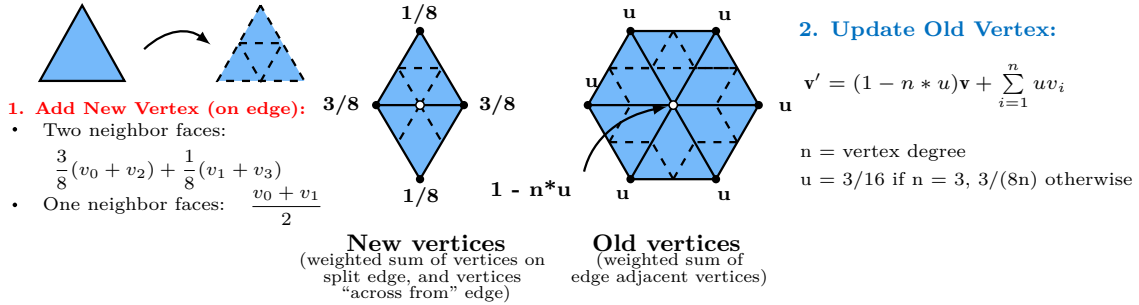
整个算法的步骤可以概括为:

1. 对每个面新增一个 face point，对每条边新增一个 edge point；
2. 按上式更新每个原顶点，得到新的 vertex point；
3. 把这些点连成新的边：每个 face point 与它所在面各条边的 edge point 相连，每个新的 vertex point 与它周围的 edge point 相连；
4. 存下新的面，原来的一个面被切成四个新面（1 old face $\rightarrow$ 4 new）。

5.2.2 Loop subdivision

循环平滑是针对于三角形网格的算法，针对三角形网格实际上就是每一次取中点一分为四，然后迭代更新。

一次迭代只做两件事：在每条边上插入一个新顶点，原来的三角形因此被切成四个小三角形；再把所有的老顶点挪到新的位置。这两件事都只用上一轮的顶点位置来计算，算完之后新顶点和老顶点合在一起构成新的三角形网格，作为下一轮的输入，如此反复迭代，网格越来越密，最后收敛到一张光滑曲面。



第一步：新增边点 (add new vertex on edge)。设被分裂的边为 (v_0, v_2) ，它两侧的三角形中各有一个“对着这条边”的顶点 (across from edge)，记为 v_1, v_3 。当这条边被两个面共用时，新顶点取

$$v_{\text{new}} = \frac{3}{8}(v_0 + v_2) + \frac{1}{8}(v_1 + v_3).$$

四个权重之和为 $\frac{3}{8} + \frac{3}{8} + \frac{1}{8} + \frac{1}{8} = 1$ ，所以新顶点仍然是老顶点的加权平均，只不过它明显更靠近自己所在的那条边的两个端点（各占 $3/8$ ），两个“对面顶点”各只出 $1/8$ 。如果这条边只有一个相邻面（边界边，one neighbor face），就直接取中点

$$v_{\text{new}} = \frac{v_0 + v_1}{2}.$$

图中菱形画的就是内部边的情形：中间的白点是要新增的顶点，左右两个 $3/8$ 是被分裂边的两个端点，上下两个 $1/8$ 则是两个三角形里对着这条边的顶点。

第二步：更新老顶点 (update old vertex)。设顶点 v 的度为 n （周围邻居顶点的个数），它的 n 个邻居顶点为 $v_1, \dots, v_n$ ，则

$$v' = (1 - nu)v + \sum_{i=1}^n uv_i, \quad u = \begin{cases} 3/16, & n = 3, \\ 3/(8n), & n \neq 3. \end{cases}$$

每个邻居分到权重 u ，顶点自己保留 $1 - nu$ ，两者相加 $(1 - nu) + nu = 1$ ，所以新位置同样是加权平均。注意当 $n \neq 3$ 时 nu 恒等于 $3/8$ ：不管周围有几个邻居，顶点让出去的权重总量是固定的，这样网

格疏密不均的地方也能过渡得比较均匀；度为 3 的顶点则单独给一个稍大的 $3/16$ 作为补偿。图中六边形画的是这一步，中心的白点是待更新的老顶点，周围一圈标 u 的是它的邻居，箭头指出的 $1 - nu$ 是它留给自己的权重。

把这两步放在一起看，迭代的过程就是：

- 新顶点只依赖上一轮的位置，老顶点也只依赖上一轮的位置，两者互不依赖，因此同一轮里所有顶点的更新可以完全并行；
- 一轮迭代之后三角形个数变成原来的四倍（每条边多出一个顶点，每个三角形被切成了四个），而且切出来的还是三角形，所以可以无限迭代下去；
- 网格一轮比一轮密，极限是一张光滑曲面；由于每次都是固定系数的线性组合，极限曲面只由最初的控制网格决定；
- 除了少数度不为 6 的奇异点（extraordinary vertex）附近只有 C^1 连续外，Loop 细分曲面的极限基本都是 C^2 连续的。

5.3 Mesh Parameterization

参数化（parameterization）就是给曲面与一块二维区域建立一一对应：网格上的每个顶点 $\mathbf{s}_i = (x_i, y_i, z_i)$ 都要在平面上找到自己的位置 $\mathbf{t}_i = (u_i, v_i)$ ，于是弯曲的曲面被“摊平”成平面上的一张网格。最常见的用途是纹理映射——把一张二维图片贴到三维模型上；世界地图也是同一件事，球面用 θ, φ 参数化成 $x = R \cos \theta \cos \varphi$ 、 $y = R \sin \theta \cos \varphi$ 、 $z = R \sin \varphi$ 之后画到纸上，就是立体投影、墨卡托投影、朗伯投影这些不同的画法。

摊平必然带来形变，所以还要说明希望保留什么几何量：等距(isometric, 保长度)、等面积(equiareal)、保角(conformal)，其中保角加等面积才是等距。一般曲面做不到等距，只有可展曲面(developable surface)能真正无失真地摊开，一般曲面只能尽量减少畸变。参数化的目标区域可以是平面(planar)、球面(spherical)或基域(simplicial)等，这里只看平面参数化。开网格(open mesh)分固定边界和自由边界两种处理方式；闭网格(closed mesh)没有边界，得先把它剪开、人为造出一条边界，再当作开网格摊平。

弹簧系统(spring system)是平面参数化里最直观的一种做法。把网格的每条边看成一根弹簧，劲度系数为 D_{ij} ；已知的是三维网格的顶点位置 $\mathbf{s}_i$ ，要求的是它们在平面上的位置 $\mathbf{t}_i$ 。把边界顶点按顺序钉在一个凸多边形上不动，让内部的顶点自由滑动，弹簧的总弹性势能就是

$$E = \frac{1}{2} \sum_{i=1}^n \sum_{j \in N_i} \frac{1}{2} D_{ij} \|\mathbf{t}_i - \mathbf{t}_j\|^2,$$

其中 N_i 是顶点 i 的邻居集合。之所以要钉住边界，是因为单看这个能量，把所有顶点叠在一起（或者任意仿射变换后的构型）能量为零，不约束就没有意义；钉住边界相当于给出“摊开”的基准。

能量最小意味着每个顶点都停在自己受力的平衡点，即对每个 $\mathbf{t}_i$ 求导得零：

$$\frac{\partial E}{\partial \mathbf{t}_i} = \sum_{j \in N_i} D_{ij} (\mathbf{t}_i - \mathbf{t}_j) = 0 \implies \sum_{j \in N_i} D_{ij} \mathbf{t}_i = \sum_{j \in N_i} D_{ij} \mathbf{t}_j \implies \mathbf{t}_i = \sum_{j \in N_i} \lambda_{ij} \mathbf{t}_j,$$

其中

$$\lambda_{ij} = D_{ij} / \sum_{k \in N_i} D_{ik}$$

是归一化后的权重, $\sum_{j \in N_i} \lambda_{ij} = 1$ 。这个结果很值得念一遍：**最优点处，每个内部顶点恰好落在它所有邻居的加权平均位置上**——邻居越“重要”（弹簧系数 D_{ij} 越大），拉得越狠。这也就是前面细分曲面里反复出现的“顶点取邻居加权平均”，只是那里权重来自工程经验，这里来自弹簧能量的最小化，并且要解出来才算数。

把每个顶点的那一行 $\mathbf{t}_i - \sum_{j \in N_i} \lambda_{ij} \mathbf{t}_j = 0$ 摞起来，就得到线性方程组 $\mathbf{A}\mathbf{t} = \mathbf{0}$ ：矩阵每行只有对角元 1 和邻居位置上的 $-\lambda_{ij}$ ，非常稀疏。 $\mathbf{A}\mathbf{t} = \mathbf{0}$ 当然有解（所有顶点取同一个值就是一解），所以要把边界顶点的行换成“已知值”再求解，写成

$$\bar{\mathbf{A}}\mathbf{t} = \mathbf{b}.$$

例如一个 9 个顶点的网格，1、2、3、4、5、6 是边界顶点，分别钉在 (1,2)、(2,3)、(3,3)、(4,2)、(3,1)、(2,1)，内部 7、8、9 待求。用均匀权重时，矩阵的前 6 行是单位阵（“这一行直接抄已知值”），后面几行是内部顶点的弹簧方程：某个内部顶点若连了 5 个邻居，这一行就是 5 个 1 加上对角元的 -5，右端向量前半填边界坐标、后半填 0。解这个稀疏线性方程组（直接求解，或者用 Gauss-Seidel 之类的迭代松弛：每一轮把每个内部顶点挪到它邻居的加权平均处）就得到了所有顶点的 (u_i, v_i) 。

权重 λ_{ij} 的取法决定参数化的质量，常见的三种是：

- **均匀权重 (average)**: $\lambda_{ij} = 1/n_i$, n_i 是顶点 i 的邻居个数。最省事，等价于把顶点放到邻居的算术平均处；只要边界是凸多边形，得到的参数化一定有效（三角形不会翻转），这就是 Tutte 嵌入。
- **调和坐标 (harmonic coordinate)**: $\lambda_{ij} = (\cot \gamma_{ij} + \cot \gamma_{ji})/2$, 其中 γ_{ij} 、 γ_{ji} 是边 (i, j) 两侧三角形中对着这条边的那两个角。它对应离散 Dirichlet 能量的最小化，更接近保角；代价是权重可能为负（钝角三角形），网格不够“Delaunay”时会出现三角形翻转。
- **均值坐标 (mean coordinate)**: $\lambda_{ij} = \left(\tan \frac{\alpha_{ij}}{2} + \tan \frac{\beta_{ji}}{2} \right) / r_{ij}$, 其中 α_{ij} 、 β_{ji} 是边两侧三角形在顶点 i 处的两个角， r_{ij} 是这条边的长度。权重恒为正，配合凸边界能保证映射是一一的。

A 数学知识补充

A.1 Fourier 变换

Fourier 变换是图像处理中的常用数学规律，图像处理中涉及的主要是二维的 Fourier 变换，所以这里只探讨二维的情况，高维的情况可以自行推广。Fourier 变换听起来很高级，但是实际上本质就是一个特殊的可逆线性变换，（在图像处理中相当于变换坐标轴），变换的目的类似于极坐标方程和直角坐标方程之间的转换。也就是对于同一个函数或者点，用不同的方式选取我们关注的信息进行表达。Fourier 变换会有计算上和理解上的好处，我们马上就会看到。

A.1.1 二维 Fourier 变换的定义

定义 A.1 (二维 Fourier 变换离散版本). 设定二维空间域中的函数为 $f(x, y)$, 则该函数的 Fourier 变换结果为

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

连续版本的只需要把求和改写成二重积分, 没有什么难度, 而且和 VCL 处理关系不大, 不予列举。现在我们需要明确 DFT 的含义。DFT 变换之后得到的函数是一个复值函数, 可以写成实部和虚部分离的形式:

$$F(u, v) = R(u, v) + jI(u, v)$$

二维 Fourier 变换的意义不太清楚, 我们回忆一维 Fourier 变换的想法。对于一个信号函数, 应当是由不同频率的正弦波叠加而成的。Fourier 变换给出的关于频率 ω 的函数说明了不同频率的波形情况。将频率的概念推广到二维频域, 则频域内的每一个点都对应了一个函数, 说明了本频率空间内的信号情况。回看上式, $\sqrt{R^2 + I^2}$, 被称之为模值, 说明了频率的强弱也就是携带的能量 (所谓高频低频指的就是这个)。我们画的“频谱”也就是指“幅值”的频谱。而该复数的辐角 $\theta = \arctan \frac{I}{R}$ 称之为相位。

A.1.2 二维 Fourier 变换的性质

Fourier 变换不仅有分析上的好处, 还有计算上的好处。首先二维 Fourier 变换是一个线性变换, 这点不难验证。从定义中可以看出, 这其实就是对二维矩阵进行了矩阵乘法。Fourier 变换对应的矩阵是 Fourier 矩阵, 定义如下:

定义 A.2 (Fourier 矩阵). 定义旋转因子:

$$\omega_N = e^{-j\frac{2\pi}{N}}$$

矩阵 $\mathbf{F}_N$ 的第 k 行、第 n 列 (索引均从 0 到 $N-1$) 的元素为:

$$\mathbf{F}_N[k, n] = \omega_N^{k \cdot n} = e^{-j\frac{2\pi}{N}kn}$$

容易验证, 这是一个酉矩阵, 因此 Fourier 变换具有酉变换的所有性质。现在请我们回忆酉变换最重要的性质:

首先是可逆性, 这意味着转换到频域之后不会有任何信息丢失。(也就是说对这个矩阵携带的信息进行了重新的分类而不改变内容)。

对于二维 Fourier 变换的逆变换有如下公式:

定理 A.3 (Fourier 变换的逆变换). 设 $f(x, y)$ 进行 Fourier 变换之后变成了 $F(u, v)$, 则逆变换形如以下表达式:

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

其次是保距性，由于是酉矩阵，变换前后点之间的距离是不会发生变化的。在研究频谱的视角下这意味着信号函数携带的总能量严格等于频域内各点所携带的总能量之和，此即著名的 Parseval 等式。能量在 VCL 的信息处理中有极为重要的应用。

定理 A.4 (Parseval 等式). Parseval 等式写作：

$$\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} |f[x, y]|^2 = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} |F[u, v]|^2$$

注：这时工程上常常采用的没有归一化的形式，严格的定义两面各自用一个常数归一化就可以。

这说明了每个频域点携带的能量，和原来每个像素点携带的能量是完全相等的。能量可以简单理解为信息量，这进一步说明 Fourier 变换前后信息没有丢失。

A.1.3 Fourier 变换的运算

Fourier 变换重要的运算性质有两个，首先是 Fourier 变换由于指数的性质容易降维拆分，在计算过程中可以直接先后计算两个一维 Fourier 变换而不需要计算二维的结构。正如以下公式：

$$F(u, v) = \sum_{x=0}^{M-1} \left(\sum_{y=0}^{N-1} f(x, y) e^{-j2\pi \frac{vy}{N}} \right) e^{-j2\pi \frac{ux}{M}}$$

首先对每行求，然后再对结果求，这样可以将计算复杂度由 $O(M^2N^2)$ 降到 $O(MN \log(MN))$

其次是 Fourier 变换可以帮助计算卷积和高阶导数。卷积和高阶导数的计算是非常复杂的，然而经过 Fourier 变换，卷积可以退化为乘法计算，可以有效降低计算复杂度。具体关系如下，其中 $*$ 表示做卷积。

$$\mathcal{F}\{f * g\} = \mathcal{F}\{f\} \mathcal{F}\{g\} \quad \mathcal{F}\left\{\frac{d^n f}{dx^n}\right\} = (j2\pi u)^n \mathcal{F}\{u\}$$

在进行复杂的卷积计算时，可以首先用 DFT 变换做乘法，然后变换回来，达到降低计算量的效果。

B Dithering 算法补充

Dithering 算法的主要难点在于各种不同的噪声。我们应当知道噪声的原理，各种噪声的数学模型和生成方法。